

Handwritten Digit Generator using Generative Adversarial Network (GAN)

Project Title: Handwritten Digit Generator using GAN

Technology: Python, TensorFlow, Keras

Dataset: MNIST Handwritten Digits Dataset

1. Project Objective

The objective of this project is to build a deep learning model capable of generating handwritten digit images using a Generative Adversarial Network (GAN). The project demonstrates how two neural networks compete with each other during training to produce realistic handwritten digits similar to those found in the MNIST dataset.

2. Introduction

Generative Adversarial Networks (GANs) are one of the most popular deep learning techniques for image generation. A GAN consists of two neural networks:

- **Generator**
- **Discriminator**

The Generator creates fake images from random noise, while the Discriminator attempts to determine whether an image is real or generated. During training, both models improve simultaneously, allowing the Generator to produce increasingly realistic handwritten digits.

The MNIST dataset contains grayscale images of handwritten digits from 0 to 9. It is widely used for learning and evaluating deep learning models.

3. Dataset

The project uses the MNIST handwritten digit dataset available in TensorFlow.

Dataset Information

- Total Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28 × 28 pixels
- Image Type: Grayscale
- Classes: Digits 0–9

4. Project Workflow

The complete workflow of the project is shown below.

1. Import required libraries.
2. Load the MNIST dataset.
3. Normalize image pixel values.
4. Create TensorFlow data pipeline.
5. Build the Generator model.
6. Build the Discriminator model.
7. Define Generator and Discriminator loss functions.
8. Configure Adam optimizers.
9. Train the GAN model.
10. Generate handwritten digit images after every epoch.
11. Save generated images.
12. Compare generated images from different epochs.
13. Save trained Generator and Discriminator models.
14. Analyze project results.

5. Model Architecture

Generator

The Generator converts a random noise vector into a handwritten digit image.

Layers used:

- Dense Layer
- Batch Normalization
- LeakyReLU
- Reshape Layer
- Conv2DTranspose Layers
- Tanh Activation

Output Size:

28 × 28 × 1

Discriminator

The Discriminator classifies whether an image is real or generated.

Layers used:

- Conv2D
- LeakyReLU
- Dropout
- Flatten
- Dense Layer

Output:

- Real Image
- Fake Image

6. Training Process

The Generator receives random noise as input and generates handwritten digit images.

The Discriminator receives both:

- Real images from the MNIST dataset
- Fake images created by the Generator

The Generator attempts to fool the Discriminator, while the Discriminator learns to correctly classify real and fake images.

Both models are trained together using Binary Crossentropy Loss and the Adam Optimizer.

During training:

- Generator Loss is minimized.
- Discriminator Loss is minimized.
- Generated images gradually improve in quality.

Generated images are saved after every training epoch to monitor learning progress.

7. Results

The GAN model successfully generated handwritten digits after training.

Observed results:

- Initial epochs produced noisy and unclear images.
- Image quality gradually improved as training progressed.
- Later epochs generated recognizable handwritten digits.
- The Generator successfully learned the distribution of handwritten digits.
- Both Generator and Discriminator models were saved for future use.

The comparison of generated images across different epochs clearly shows continuous improvement in image quality.

8. Challenges Faced

Some common challenges encountered during GAN training include:

- Training instability.
- Maintaining balance between Generator and Discriminator.
- Noisy outputs during early epochs.
- Slow convergence.
- Requirement of multiple training epochs for realistic image generation.
- Occasional generation of similar-looking digits due to limited diversity.

These challenges are common in GAN-based image generation projects and can be reduced through careful tuning of hyperparameters and extended training.

9. Conclusion

This project successfully implemented a Generative Adversarial Network using TensorFlow and the MNIST dataset.

The Generator learned to create realistic handwritten digit images from random noise, while the Discriminator learned to distinguish generated images from real handwritten digits.

The quality of generated images improved significantly throughout the training process, demonstrating the effectiveness of adversarial learning. This project provided practical experience in deep learning, neural network training, image generation, and GAN architecture.

Overall, the project achieved its objective of generating handwritten digits using a GAN and demonstrated the complete workflow from data preprocessing to model training, image generation, result analysis, and model saving.

10. Future Improvements

The project can be enhanced by implementing the following improvements:

- Train the model for a larger number of epochs.
- Use Deep Convolutional GAN (DCGAN) architecture.
- Apply Wasserstein GAN (WGAN) for improved training stability.
- Generate higher-resolution handwritten digits.
- Add quantitative evaluation metrics such as FID or Inception Score.
- Extend the model to generate colored images or other object categories.

Tools and Technologies

- Python
- TensorFlow
- Keras
- NumPy
- Matplotlib
- Google Colab
- MNIST Dataset